

Math 470: Communications and Cryptography



J. Maurice Rojas
September 25, 2023

Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$,



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$,



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$, i.e., we just need to do *one* doubling to get $[2](14, 21)$,



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$, i.e., we just need to do *one* doubling to get $[2](14, 21)$, and then *three* more to get $[16](14, 21) = [2][2][2][2](14, 21)$,



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$, i.e., we just need to do *one* doubling to get $[2](14, 21)$, and then *three* more to get $[16](14, 21) = [2][2][2][2](14, 21)$, and then just *one* more addition to compute $[18](14, 21) = [16](14, 21) \oplus [2](14, 21)$.



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$, i.e., we just need to do *one* doubling to get $[2](14, 21)$, and then *three* more to get $[16](14, 21) = [2][2][2][2](14, 21)$, and then just *one* more addition to compute $[18](14, 21) = [16](14, 21) \oplus [2](14, 21)$.

Carrying this out, we get:



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$, i.e., we just need to do *one* doubling to get $[2](14, 21)$, and then *three* more to get $[16](14, 21) = [2][2][2][2](14, 21)$, and then just *one* more addition to compute $[18](14, 21) = [16](14, 21) \oplus [2](14, 21)$.

Carrying this out, we get: $[2](14, 21) = (50, 6)$,



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$, i.e., we just need to do *one* doubling to get $[2](14, 21)$, and then *three* more to get $[16](14, 21) = [2][2][2][2](14, 21)$, and then just *one* more addition to compute $[18](14, 21) = [16](14, 21) \oplus [2](14, 21)$.

Carrying this out, we get: $[2](14, 21) = (50, 6)$,
 $[4](14, 21) = (4, 11)$,



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$, i.e., we just need to do *one* doubling to get $[2](14, 21)$, and then *three* more to get $[16](14, 21) = [2][2][2][2](14, 21)$, and then just *one* more addition to compute $[18](14, 21) = [16](14, 21) \oplus [2](14, 21)$.

Carrying this out, we get: $[2](14, 21) = (50, 6)$,
 $[4](14, 21) = (4, 11)$, $[8](14, 21) = (7, 20)$,



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$, i.e., we just need to do *one* doubling to get $[2](14, 21)$, and then *three* more to get $[16](14, 21) = [2][2][2][2](14, 21)$, and then just *one* more addition to compute $[18](14, 21) = [16](14, 21) \oplus [2](14, 21)$.

Carrying this out, we get: $[2](14, 21) = (50, 6)$,
 $[4](14, 21) = (4, 11)$, $[8](14, 21) = (7, 20)$,
 $[16](14, 21) = (5, 8)$.



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$, i.e., we just need to do *one* doubling to get $[2](14, 21)$, and then *three* more to get $[16](14, 21) = [2][2][2][2](14, 21)$, and then just *one* more addition to compute $[18](14, 21) = [16](14, 21) \oplus [2](14, 21)$.

Carrying this out, we get: $[2](14, 21) = (50, 6)$,
 $[4](14, 21) = (4, 11)$, $[8](14, 21) = (7, 20)$,
 $[16](14, 21) = (5, 8)$. Finally, we get



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

First observe that $18 = 2^4 + 2^1$, so we can re-express $[18](14, 21)$ as $[2][2][2][2](14, 21) \oplus [2](14, 21)$, i.e., we just need to do *one* doubling to get $[2](14, 21)$, and then *three* more to get $[16](14, 21) = [2][2][2][2](14, 21)$, and then just *one* more addition to compute $[18](14, 21) = [16](14, 21) \oplus [2](14, 21)$.

Carrying this out, we get: $[2](14, 21) = (50, 6)$,
 $[4](14, 21) = (4, 11)$, $[8](14, 21) = (7, 20)$,
 $[16](14, 21) = (5, 8)$. Finally, we get

$$[18](14, 21) = (57, 52).$$



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$,
using just 5 elliptic curve additions.

Some Maple code that works:



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

Some Maple code that works:

```
# let's do the addition law on any curve in short weierstrass form, i.e.,
# y^2 = x^3 + Ax + B, where also insert reduction mod p, for p some prime..

# bachet over F_{59}: y^2 = x^3 - 2
p:=59:
A:= 0:
B:= -2:

f:=(x,y) -> (y^2-(x^3+A*x+B) mod p):

lam:=proc(x1,y1,x2,y2)
  if (x1<>x2) or (y1<>y2) then
    if (x1=x2) and (y1=-y2) then
      return infinity:
    else
      return (y2-y1)/(x2-x1):
    end:
  else
    return (3 * x1^2 + A)/(2 * y1):
  end:
end proc:

x3:=(x1,y1,x2,y2) -> ((lam(x1,y1,x2,y2)^2-x1-x2) mod p):
y3:=(x1,y1,x2,y2) -> ((lam(x1,y1,x2,y2)*(x1-x3(x1,y1,x2,y2))-y1) mod p):
```



Answers to Ungraded Quiz #41!

Please compute $[18](14, 21)$ in $E(\mathbb{F}_{59})$, where $E: y^2 := x^3 - 2$, using just 5 elliptic curve additions.

Some Maple code that works:

```
# let's do the addition law on any curve in short weierstrass form, i.e.,
# y^2 = x^3 + Ax + B, where also insert reduction mod p, for p some prime.

# bachet over F_{59}: y^2 = x^3 - 2
p:=59:
A:= 0:
B:= -2:

f:=(x,y) -> (y^2-(x^3+A*x+B) mod p):

lam:=proc(x1,y1,x2,y2)
  if (x1<>x2) or (y1<>y2) then
    if (x1=x2) and (y1=-y2) then
      return infinity:
    else
      return (y2-y1)/(x2-x1):
    end:
  else
    return (3 * x1^2 + A)/(2 * y1):
  end:
end proc:

read("ecadd.map");
p2x,p2y:=x3(14,21,14,21),y3(14,21,14,21);
p4x,p4y:=x3(p2x,p2y,p2x,p2y),y3(p2x,p2y,p2x,p2y);
p8x,p8y:=x3(p4x,p4y,p4x,p4y),y3(p4x,p4y,p4x,p4y);
p16x,p16y:= x3(p8x,p8y,p8x,p8y),y3(p8x,p8y,p8x,p8y);
p18x,p18y:= x3(p16x,p16y,p2x,p2y),y3(p16x,p16y,p2x,p2y);

x3:=(x1,y1,x2,y2) -> ((lam(x1,y1,x2,y2)^2-x1-x2) mod p):
y3:=(x1,y1,x2,y2) -> ((lam(x1,y1,x2,y2)*(x1-x3(x1,y1,x2,y2))-y1) mod p):
```



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols...



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985,



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).

Elliptic Curve Crypto (ECC) started being used widely around 2004.



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).

Elliptic Curve Crypto (ECC) started being used widely around 2004.

However, even earlier, in 1984,



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).

Elliptic Curve Crypto (ECC) started being used widely around 2004.

However, even earlier, in 1984, Hendrik W. Lenstra, Jr. invented *ECM*:



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).

Elliptic Curve Crypto (ECC) started being used widely around 2004.

However, even earlier, in 1984, Hendrik W. Lenstra, Jr. invented *ECM*: A sub-exponential integer factorization method using elliptic curves!



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).

Elliptic Curve Crypto (ECC) started being used widely around 2004.

However, even earlier, in 1984, Hendrik W. Lenstra, Jr. invented *ECM*: A sub-exponential integer factorization method using elliptic curves!

ECM is still the best for finding “mid-size” prime divisors.



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).

1985: Scott Vanstone and Ron Mullin start a cryptological company called Certicom.



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).

1985: Scott Vanstone and Ron Mullin start a cryptological company called Certicom. The interactions between crypto, academia, profit, and patents are complicated...



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).

1992: Leonard Adleman and Min-Deh Huang,



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).

1992: Leonard Adleman and Min-Deh Huang, based on 1986 work of Shafi Goldwasser and Joe Killian,



A little history...

We noted recently that it's ultimately “easy” to derive elliptic curve analogues of certain discrete log-based cryptographic protocols... This was actually an innovation found independently, in 1985, by Neal Koblitz (U Washington) and Victor Miller (IBM).

1992: Leonard Adleman and Min-Deh Huang, based on 1986 work of Shafi Goldwasser and Joe Killian, prove that elliptic curves can be used to prove that primality can be decided in randomized polynomial-time!



The key idea behind ECM is parallel to the key idea behind Pollard's $(p - 1)$ -method:



The key idea behind ECM is parallel to the key idea behind Pollard's $(p - 1)$ -method: Create a random recurrence that (hopefully) reveals a non-trivial factor after *few* iterations.



The key idea behind ECM is parallel to the key idea behind Pollard's $(p - 1)$ -method: Create a random recurrence that (hopefully) reveals a non-trivial factor after *few* iterations.

In particular, ECM computes $[k!]P$ for *random* P



The key idea behind ECM is parallel to the key idea behind Pollard's $(p - 1)$ -method: Create a random recurrence that (hopefully) reveals a non-trivial factor after *few* iterations.

In particular, ECM computes $[k!]P$ for *random* P (in a random elliptic curve E over $\mathbb{Z}/(N)$),



The key idea behind ECM is parallel to the key idea behind Pollard's $(p - 1)$ -method: Create a random recurrence that (hopefully) reveals a non-trivial factor after *few* iterations.

In particular, ECM computes $[k!]P$ for *random* P (in a random elliptic curve E over $\mathbb{Z}/(N)$), while Pollard's method computes $a^{k!} \bmod N$ for random a



The key idea behind ECM is parallel to the key idea behind Pollard's $(p - 1)$ -method: Create a random recurrence that (hopefully) reveals a non-trivial factor after *few* iterations.

In particular, ECM computes $[k!]P$ for *random* P (in a random elliptic curve E over $\mathbb{Z}/(N)$), while Pollard's method computes $a^{k!} \bmod N$ for random a (in $(\mathbb{Z}/(N))^*$).



The key idea behind ECM is parallel to the key idea behind Pollard's $(p - 1)$ -method: Create a random recurrence that (hopefully) reveals a non-trivial factor after *few* iterations.

In particular, ECM computes $[k!]P$ for *random* P (in a random elliptic curve E over $\mathbb{Z}/(N)$), while Pollard's method computes $a^{k!} \bmod N$ for random a (in $(\mathbb{Z}/(N))^*$).

However, using the group $E(\mathbb{Z}/(N))$



The key idea behind ECM is parallel to the key idea behind Pollard's $(p - 1)$ -method: Create a random recurrence that (hopefully) reveals a non-trivial factor after *few* iterations.

In particular, ECM computes $[k!]P$ for *random* P (in a random elliptic curve E over $\mathbb{Z}/(N)$), while Pollard's method computes $a^{k!} \bmod N$ for random a (in $(\mathbb{Z}/(N))^*$).

However, using the group $E(\mathbb{Z}/(N))$ (for *many* random $E!$),



The key idea behind ECM is parallel to the key idea behind Pollard's $(p - 1)$ -method: Create a random recurrence that (hopefully) reveals a non-trivial factor after *few* iterations.

In particular, ECM computes $[k!]P$ for *random* P (in a random elliptic curve E over $\mathbb{Z}/(N)$), while Pollard's method computes $a^{k!} \bmod N$ for random a (in $(\mathbb{Z}/(N))^*$).

However, using the group $E(\mathbb{Z}/(N))$ (for *many* random E !), instead of $(\mathbb{Z}/(N))^*$,



The key idea behind ECM is parallel to the key idea behind Pollard's $(p - 1)$ -method: Create a random recurrence that (hopefully) reveals a non-trivial factor after *few* iterations.

In particular, ECM computes $[k!]P$ for *random* P (in a random elliptic curve E over $\mathbb{Z}/(N)$), while Pollard's method computes $a^{k!} \bmod N$ for random a (in $(\mathbb{Z}/(N))^*$).

However, using the group $E(\mathbb{Z}/(N))$ (for *many* random E !), instead of $(\mathbb{Z}/(N))^*$, one can amplify the chances of success!



...

In particular, ECM computes $[k!]P$ for *random* P (in a random elliptic curve E over $\mathbb{Z}/(N)$), while Pollard's method computes $a^{k!} \bmod N$ for random a (in $(\mathbb{Z}/(N))^*$).

However, using the group $E(\mathbb{Z}/(N))$ (for *many* random E !), instead of $(\mathbb{Z}/(N))^*$, one can amplify the chances of success!

Note: [HPS14, Ch. 6, Sec. 6.6] does the examples of 187



...

In particular, ECM computes $[k!]P$ for *random* P (in a random elliptic curve E over $\mathbb{Z}/(N)$), while Pollard's method computes $a^{k!} \bmod N$ for random a (in $(\mathbb{Z}/(N))^*$).

However, using the group $E(\mathbb{Z}/(N))$ (for *many* random E !), instead of $(\mathbb{Z}/(N))^*$, one can amplify the chances of success!

Note: [HPS14, Ch. 6, Sec. 6.6] does the examples of 187 and 6887.



...

In particular, ECM computes $[k!]P$ for *random* P (in a random elliptic curve E over $\mathbb{Z}/(N)$), while Pollard's method computes $a^{k!} \bmod N$ for random a (in $(\mathbb{Z}/(N))^*$).

However, using the group $E(\mathbb{Z}/(N))$ (for *many* random E !), instead of $(\mathbb{Z}/(N))^*$, one can amplify the chances of success!

Note: [HPS14, Ch. 6, Sec. 6.6] does the examples of 187 and 6887. We'll do a slightly a less trivial example...



...

However, using the group $E(\mathbb{Z}/(N))$ (for *many* random $E!$), instead of $(\mathbb{Z}/(N))^*$, one can amplify the chances of success!

Note: [HPS14, Ch. 6, Sec. 6.6] does the examples of 187 and 6887. We'll do a slightly a less trivial example...

But first, let's warm up with observations on



...

However, using the group $E(\mathbb{Z}/(N))$ (for *many* random $E!$), instead of $(\mathbb{Z}/(N))^*$, one can amplify the chances of success!

Note: [HPS14, Ch. 6, Sec. 6.6] does the examples of 187 and 6887. We'll do a slightly a less trivial example...

But first, let's warm up with observations on (1) binary expansion with *subtraction* and



...

However, using the group $E(\mathbb{Z}/(N))$ (for *many* random $E!$), instead of $(\mathbb{Z}/(N))^*$, one can amplify the chances of success!

Note: [HPS14, Ch. 6, Sec. 6.6] does the examples of 187 and 6887. We'll do a slightly a less trivial example...

But first, let's warm up with observations on

- (1) binary expansion with *subtraction* and
- (2) computing factorials efficiently...



Augmenting binary with minus?

At a low level, it doesn't make sense to refine binary expansion:



Augmenting binary with minus?

At a low level, it doesn't make sense to refine binary expansion: Binary encoding yields a standard hardware method to represent numbers,



Augmenting binary with minus?

At a low level, it doesn't make sense to refine binary expansion: Binary encoding yields a standard hardware method to represent numbers, and observations like $254 = 2^8 - 2^1$



Augmenting binary with minus?

At a low level, it doesn't make sense to refine binary expansion: Binary encoding yields a standard hardware method to represent numbers, and observations like $254 = 2^8 - 2^1$ (which involves just 2 powers of 2,



Augmenting binary with minus?

At a low level, it doesn't make sense to refine binary expansion: Binary encoding yields a standard hardware method to represent numbers, and observations like $254 = 2^8 - 2^1$ (which involves just 2 powers of 2, instead of the 7 for the binary expansion 2)



Augmenting binary with minus?

At a low level, it doesn't make sense to refine binary expansion: Binary encoding yields a standard hardware method to represent numbers, and observations like $254 = 2^8 - 2^1$ (which involves just 2 powers of 2, instead of the 7 for the binary expansion 2) are *not* enough to justify remaking hardware...



Augmenting binary with minus?

At a low level, it doesn't make sense to refine binary expansion: Binary encoding yields a standard hardware method to represent numbers, and observations like $254 = 2^8 - 2^1$ (which involves just 2 powers of 2, instead of the 7 for the binary expansion 2) are *not* enough to justify remaking hardware...

Likewise, in the binary method for powering,



Augmenting binary with minus?

At a low level, it doesn't make sense to refine binary expansion: Binary encoding yields a standard hardware method to represent numbers, and observations like $254 = 2^8 - 2^1$ (which involves just 2 powers of 2, instead of the 7 for the binary expansion 2) are *not* enough to justify remaking hardware...

Likewise, in the binary method for powering, one *could* try to make use of subtraction to make powering more efficient:



Augmenting binary with minus?

At a low level, it doesn't make sense to refine binary expansion: Binary encoding yields a standard hardware method to represent numbers, and observations like $254 = 2^8 - 2^1$ (which involves just 2 powers of 2, instead of the 7 for the binary expansion 2) are *not* enough to justify remaking hardware...

Likewise, in the binary method for powering, one *could* try to make use of subtraction to make powering more efficient: For instance, computing $a^{254} \bmod N$ via $a^{2^8} / a^{2^1} \bmod N$



Augmenting binary with minus?

At a low level, it doesn't make sense to refine binary expansion: Binary encoding yields a standard hardware method to represent numbers, and observations like $254 = 2^8 - 2^1$ (which involves just 2 powers of 2, instead of the 7 for the binary expansion 2) are *not* enough to justify remaking hardware...

Likewise, in the binary method for powering, one *could* try to make use of subtraction to make powering more efficient: For instance, computing $a^{254} \bmod N$ via $a^{2^8} / a^{2^1} \bmod N$ (using just 8 squarings and 1 division),



Augmenting binary with minus?

...observations like $254 = 2^8 - 2^1$ (which involves just 2 powers of 2, instead of the 7 for the binary expansion 2) are *not* enough to justify remaking hardware...

Likewise, in the binary method for powering, one *could* try to make use of subtraction to make powering more efficient: For instance, computing $a^{254} \bmod N$ via $a^{2^8}/a^{2^1} \bmod N$ (using just 8 squarings and 1 division), instead of the usual $a^{2^7} a^{2^6} a^{2^5} a^{2^4} a^{2^3} a^{2^2} a^{2^1} \bmod N$ (which involves 7 squarings and 6 multiplications).



Augmenting binary with minus?

Likewise, in the binary method for powering, one *could* try to make use of subtraction to make powering more efficient: For instance, computing $a^{254} \bmod N$ via $a^{2^8}/a^{2^1} \bmod N$ (using just 8 squarings and 1 division), instead of the usual $a^{2^7} a^{2^6} a^{2^5} a^{2^4} a^{2^3} a^{2^2} a^{2^1} \bmod N$ (which involves 7 squarings and 6 multiplications). However, division $\bmod N$ is *slightly* more expensive than multiplication $\bmod N$.



Likewise, in the binary method for powering, one *could* try to make use of subtraction to make powering more efficient: For instance, computing $a^{254} \bmod N$ via $a^{2^8} / a^{2^1} \bmod N$ (using just 8 squarings and 1 division), instead of the usual $a^{2^7} a^{2^6} a^{2^5} a^{2^4} a^{2^3} a^{2^2} a^{2^1} \bmod N$ (which involves 7 squarings and 6 multiplications). However, division $\bmod N$ is *slightly* more expensive than multiplication $\bmod N$. So a proper speed-up would require some extra overhead for accounting,



Likewise, in the binary method for powering, one *could* try to make use of subtraction to make powering more efficient: For instance, computing $a^{254} \bmod N$ via $a^{2^8}/a^{2^1} \bmod N$ (using just 8 squarings and 1 division), instead of the usual $a^{2^7} a^{2^6} a^{2^5} a^{2^4} a^{2^3} a^{2^2} a^{2^1} \bmod N$ (which involves 7 squarings and 6 multiplications). However, division $\bmod N$ is *slightly* more expensive than multiplication $\bmod N$. So a proper speed-up would require some extra overhead for accounting, which could cost as much as the speed-up!



HOWEVER: For *elliptic curve addition*,



HOWEVER: For *elliptic curve addition*, addition and subtraction cost essentially the same!:



HOWEVER: For *elliptic curve addition*, addition and subtraction cost essentially the same!

$P - Q$ is,



HOWEVER: For *elliptic curve addition*, addition and subtraction cost essentially the same!

$P - Q$ is, by definition,



HOWEVER: For *elliptic curve addition*, addition and subtraction cost essentially the same!

$P - Q$ is, by definition, $P + (-Q)$,



HOWEVER: For *elliptic curve addition*, addition and subtraction cost essentially the same!:

$P - Q$ is, by definition, $P + (-Q)$, and computing $-Q$ simply means taking the negative of the y -coordinate of Q !



HOWEVER: For *elliptic curve addition*, addition and subtraction cost essentially the same!:

$P - Q$ is, by definition, $P + (-Q)$, and computing $-Q$ simply means taking the negative of the y -coordinate of Q ! **when in Weierstrass form!*



HOWEVER: For *elliptic curve addition*, addition and subtraction cost essentially the same!:

$P - Q$ is, by definition, $P + (-Q)$, and computing $-Q$ simply means taking the negative of the y -coordinate of Q ! **when in Weierstrass form!*

More to the point, one can *prove* that one gains a $\approx 12.5\%$ speed-up on average!:



HOWEVER: For *elliptic curve addition*, addition and subtraction cost essentially the same!:

$P - Q$ is, by definition, $P + (-Q)$, and computing $-Q$ simply means taking the negative of the y -coordinate of Q ! **when in Weierstrass form!*

More to the point, one can *prove* that one gains a $\approx 12.5\%$ speed-up on average!: See [HPS14, Prop. 6.18].



HOWEVER: For *elliptic curve addition*, addition and subtraction cost essentially the same!:

$P - Q$ is, by definition, $P + (-Q)$, and computing $-Q$ simply means taking the negative of the y -coordinate of Q ! **when in Weierstrass form!*

More to the point, one can *prove* that one gains a $\approx 12.5\%$ speed-up on average!: See [HPS14, Prop. 6.18]. For commercial-grade crypto, such speed-ups matter!



Factorial?

The complexity of computing factorials is a surprisingly deep problem!



Factorial?

The complexity of computing factorials is a surprisingly deep problem! ‘

- ① Low *circuit* complexity for $k!$ implies that integer factoring can be done quickly! [Shamir, 1977].



Factorial?

The complexity of computing factorials is a surprisingly deep problem! ‘

- ① Low *circuit* complexity for $k!$ implies that integer factoring can be done quickly! [Shamir, 1977].
- ② Computing $a^{k!} \bmod N$ efficiently is a key step in Pollard's $(p-1)$ -Method integer factoring method!



Factorial?

The complexity of computing factorials is a surprisingly deep problem! ‘

- ① Low *circuit* complexity for $k!$ implies that integer factoring can be done quickly! [Shamir, 1977].
- ② Computing $a^{k!} \bmod N$ efficiently is a key step in Pollard's $(p-1)$ -Method integer factoring method!
- ③ Computing $[k!]P$ in $E(\mathbb{Z}/(N))$ efficiently is a key step in *Lenstra's ECM Method* for integer factoring!



Factorial example on an elliptic curve?

Consider computing $[399!]P$ in $E(\mathbb{Z}/(N))$:



Factorial example on an elliptic curve?

Consider computing $[399!]P$ in $E(\mathbb{Z}/(N))$: The worst thing to do is to compute $399!$,



Factorial example on an elliptic curve?

Consider computing $[399!]P$ in $E(\mathbb{Z}/(N))$: The worst thing to do is to compute $399!$, since it is **MASSIVE!**:



Factorial example on an elliptic curve?

Consider computing $[399!]P$ in $E(\mathbb{Z}/(N))$: The worst thing to do is to compute $399!$, since it is **MASSIVE!**:

16008630711655973815586992579875751462675645
75650073986467111148570059929229670785906961
96618658161690735876437589642027120745407208
79358807240497161700749484335413537709540606
61548558807676156108125377861216772266569347
87295293329889991101773874178363226192550806
08727802699398320198775386343166812906969472
50233744094142758158758288349133746709670783
48380060934470394466978765779646756545675424
54935015745756327147824586540568076139584880
18990287632555902170260832431379871316860805
81096674871056010581499513879026589855942403
49807979283515964749134492536956801651580054
34486800258033915615345226948557614934017489
18989590240396787824784555716446448873404044
13620113305501956454600212109103897807363568
84620088959362950566897501534989003639880153
180279822952625812275200000000000000000000000
00
00



Factorial example on an elliptic curve?

Consider computing $[399!]P$ in $E(\mathbb{Z}/(N))$: The worst thing to do is to compute $399!$, since it is **MASSIVE!**:

16008630711655973815586992579875751462675645
75650073986467111148570059929229670785906961
96618658161690735876437589642027120745407...

...and involves 399 multiplications of numbers with up to 2878 bits,



Factorial example on an elliptic curve?

Consider computing $[399!]P$ in $E(\mathbb{Z}/(N))$: The worst thing to do is to compute $399!$, since it is **MASSIVE!**:

16008630711655973815586992579875751462675645
75650073986467111148570059929229670785906961
96618658161690735876437589642027120745407...

...and involves 399 multiplications of numbers with up to 2878 bits, and keeping 2877 doublings in memory.



Factorial example on an elliptic curve?

Consider computing $[399!]P$ in $E(\mathbb{Z}/(N))$: The worst thing to do is to compute $399!$, since it is **MASSIVE!**:

16008630711655973815586992579875751462675645
75650073986467111148570059929229670785906961
96618658161690735876437589642027120745407...

...and involves 399 multiplications of numbers with up to 2878 bits, and keeping 2877 doublings in memory.

However, if we simply use the obvious recurrence implied by:



Factorial example on an elliptic curve?

Consider computing $[399!]P$ in $E(\mathbb{Z}/(N))$: The worst thing to do is to compute $399!$, since it is **MASSIVE!**:

16008630711655973815586992579875751462675645
75650073986467111148570059929229670785906961
96618658161690735876437589642027120745407...

...and involves 399 multiplications of numbers with up to 2878 bits, and keeping 2877 doublings in memory.

However, if we simply use the obvious recurrence implied by: $[399]([398](\cdots([2]P))\cdots)$,



Factorial example on an elliptic curve?

Consider computing $[399!]P$ in $E(\mathbb{Z}/(N))$: The worst thing to do is to compute $399!$, since it is **MASSIVE!**:

16008630711655973815586992579875751462675645
75650073986467111148570059929229670785906961
96618658161690735876437589642027120745407...

...and involves 399 multiplications of numbers with up to 2878 bits, and keeping 2877 doublings in memory.

However, if we simply use the obvious recurrence implied by: $[399]([398](\cdots([2]P))\cdots)$, then we never multiply numbers with more than 9 bits,



Factorial example on an elliptic curve?

Consider computing $[399!]P$ in $E(\mathbb{Z}/(N))$: The worst thing to do is to compute $399!$, since it is **MASSIVE!**:

16008630711655973815586992579875751462675645
75650073986467111148570059929229670785906961
96618658161690735876437589642027120745407...

...and involves 399 multiplications of numbers with up to 2878 bits, and keeping 2877 doublings in memory.

However, if we simply use the obvious recurrence implied by: $[399]([398](\dots([2]P))\dots)$, then we never multiply numbers with more than 9 bits, and we need no more than 9 doublings in memory at a time!



Factorials on an elliptic curve!

In summary...

If you compute $[k!]P$ on $E(\mathbb{Z}/(N))$



Factorials on an elliptic curve!

In summary...

If you compute $[k!]P$ on $E(\mathbb{Z}/(N))$ by fully expanding $k!$ first,



In summary...

If you compute $[k!]P$ on $E(\mathbb{Z}/(N))$ by fully expanding $k!$ first, then you are doomed to consuming $\Omega(k \log k)$ bits worth of memory.



In summary...

If you compute $[k!]P$ on $E(\mathbb{Z}/(N))$ by fully expanding $k!$ first, then you are doomed to consuming $\Omega(k \log k)$ bits worth of memory. (And doing $k^{\omega(1)}$ bit operations unless major conjectures are resolved!)



In summary...

If you compute $[k!]P$ on $E(\mathbb{Z}/(N))$ by fully expanding $k!$ first, then you are doomed to consuming $\Omega(k \log k)$ bits worth of memory. (And doing $k^{\omega(1)}$ bit operations unless major conjectures are resolved!)

However, if you compute $m_2 := [2]P$,



In summary...

If you compute $[k!]P$ on $E(\mathbb{Z}/(N))$ by fully expanding $k!$ first, then you are doomed to consuming $\Omega(k \log k)$ bits worth of memory. (And doing $k^{\omega(1)}$ bit operations unless major conjectures are resolved!)

However, if you compute $m_2 := [2]P$, $m_3 := [3]m_2$,



In summary...

If you compute $[k!]P$ on $E(\mathbb{Z}/(N))$ by fully expanding $k!$ first, then you are doomed to consuming $\Omega(k \log k)$ bits worth of memory. (And doing $k^{\omega(1)}$ bit operations unless major conjectures are resolved!)

However, if you compute $m_2 := [2]P$, $m_3 := [3]m_2$,
...



In summary...

If you compute $[k!]P$ on $E(\mathbb{Z}/(N))$ by fully expanding $k!$ first, then you are doomed to consuming $\Omega(k \log k)$ bits worth of memory. (And doing $k^{\omega(1)}$ bit operations unless major conjectures are resolved!)

However, if you compute $m_2 := [2]P$, $m_3 := [3]m_2$, $\dots$, $m_k := [k]m_{k-1}$ instead,



In summary...

If you compute $[k!]P$ on $E(\mathbb{Z}/(N))$ by fully expanding $k!$ first, then you are doomed to consuming $\Omega(k \log k)$ bits worth of memory. (And doing $k^{\omega(1)}$ bit operations unless major conjectures are resolved!)

However, if you compute $m_2 := [2]P$, $m_3 := [3]m_2$, $\dots$, $m_k := [k]m_{k-1}$ instead, then you never need more than $\boxed{O(\log k)}$ bits worth of memory,



In summary...

If you compute $[k!]P$ on $E(\mathbb{Z}/(N))$ by fully expanding $k!$ first, then you are doomed to consuming $\Omega(k \log k)$ bits worth of memory. (And doing $k^{\omega(1)}$ bit operations unless major conjectures are resolved!)

However, if you compute $m_2 := [2]P$, $m_3 := [3]m_2$, $\dots$, $m_k := [k]m_{k-1}$ instead, then you never need more than $O(\log k)$ bits worth of memory, and you do $k^{1+o(1)}$ bit operations.



(UNGRADED) Quiz #42!

Consider $N := 263629786661$



(UNGRADED) Quiz #42!

Consider $N := 263629786661$ and observe that picking $(A, a, b) := (1, 2, 3)$



(UNGRADED) Quiz #42!

Consider $N := 263629786661$ and observe that picking $(A, a, b) := (1, 2, 3)$ makes $P := (a, b)$ a point on the elliptic curve E ,



(UNGRADED) Quiz #42!

Consider $N := 263629786661$ and observe that picking $(A, a, b) := (1, 2, 3)$ makes $P := (a, b)$ a point on the elliptic curve E , over $\mathbb{Z}/(N)$,



(UNGRADED) Quiz #42!

Consider $N := 263629786661$ and observe that picking $(A, a, b) := (1, 2, 3)$ makes $P := (a, b)$ a point on the elliptic curve E , over $\mathbb{Z}/(N)$, defined by $y^2 = x^3 + Ax + B$,



(UNGRADED) Quiz #42!

Consider $N := 263629786661$ and observe that picking $(A, a, b) := (1, 2, 3)$ makes $P := (a, b)$ a point on the elliptic curve E , over $\mathbb{Z}/(N)$, defined by $y^2 = x^3 + Ax + B$, provided we set $B := b^2 - a^3 - Aa \bmod N = -1$.



(UNGRADED) Quiz #42!

Consider $N := 263629786661$ and observe that picking $(A, a, b) := (1, 2, 3)$ makes $P := (a, b)$ a point on the elliptic curve E , over $\mathbb{Z}/(N)$, defined by $y^2 = x^3 + Ax + B$, provided we set $B := b^2 - a^3 - Aa \bmod N = -1$.

This $N := 263629786661$ turns out to be an RSA integer



(UNGRADED) Quiz #42!

Consider $N := 263629786661$ and observe that picking $(A, a, b) := (1, 2, 3)$ makes $P := (a, b)$ a point on the elliptic curve E , over $\mathbb{Z}/(N)$, defined by $y^2 = x^3 + Ax + B$, provided we set $B := b^2 - a^3 - Aa \bmod N = -1$.

This $N := 263629786661$ turns out to be an RSA integer and computing $[k!]P$ in $E(\mathbb{Z}/(N))$,



(UNGRADED) Quiz #42!

Consider $N := 263629786661$ and observe that picking $(A, a, b) := (1, 2, 3)$ makes $P := (a, b)$ a point on the elliptic curve E , over $\mathbb{Z}/(N)$, defined by $y^2 = x^3 + Ax + B$, provided we set $B := b^2 - a^3 - Aa \bmod N = -1$.

This $N := 263629786661$ turns out to be an RSA integer and computing $[k!]P$ in $E(\mathbb{Z}/(N))$, for “small” k ,



(UNGRADED) Quiz #42!

Consider $N := 263629786661$ and observe that picking $(A, a, b) := (1, 2, 3)$ makes $P := (a, b)$ a point on the elliptic curve E , over $\mathbb{Z}/(N)$, defined by $y^2 = x^3 + Ax + B$, provided we set $B := b^2 - a^3 - Aa \bmod N = -1$.

This $N := 263629786661$ turns out to be an RSA integer and computing $[k!]P$ in $E(\mathbb{Z}/(N))$, for “small” k , will in fact reveal a factor of N ...



Consider $N := 263629786661$ and the point $P := (2, 3)$ in the curve $E : y^2 = x^3 + x - 1$ over $\mathbb{Z}/(N)\dots$

- a. What is the smallest positive k such that $[k!]P$ becomes *ill-defined*?



(UNGRADED) Quiz #42!

Consider $N := 263629786661$ and the point $P := (2, 3)$ in the curve $E : y^2 = x^3 + x - 1$ over $\mathbb{Z}/(N)\dots$

- a. What is the smallest positive k such that $[k!]P$ becomes *ill-defined*?

Hint: This boils down to when λ



Consider $N := 263629786661$ and the point $P := (2, 3)$ in the curve $E : y^2 = x^3 + x - 1$ over $\mathbb{Z}/(N) \dots$

- a. What is the smallest positive k such that $[k!]P$ becomes *ill-defined*?

Hint: This boils down to when λ (in the addition law for curves in short Weierstrass form)



Consider $N := 263629786661$ and the point $P := (2, 3)$ in the curve $E : y^2 = x^3 + x - 1$ over $\mathbb{Z}/(N) \dots$

1. a. What is the smallest positive k such that $[k!]P$ becomes *ill-defined*?

Hint: This boils down to when λ (in the addition law for curves in short Weierstrass form) asks for an inverse that doesn't exist mod N .



Consider $N := 263629786661$ and the point $P := (2, 3)$ in the curve $E : y^2 = x^3 + x - 1$ over $\mathbb{Z}/(N)\dots$

1. a. What is the smallest positive k such that $[k!]P$ becomes *ill-defined*?
- b. Your *failed* computation of $[k!]P$ should reveal a factor of N :



Consider $N := 263629786661$ and the point $P := (2, 3)$ in the curve $E : y^2 = x^3 + x - 1$ over $\mathbb{Z}/(N)\dots$

1. a. What is the smallest positive k such that $[k!]P$ becomes *ill-defined*?
- b. Your *failed* computation of $[k!]P$ should reveal a factor of N : What is the factor?



Consider $N := 263629786661$ and the point $P := (2, 3)$ in the curve $E : y^2 = x^3 + x - 1$ over $\mathbb{Z}/(N)$...

1. a. What is the smallest positive k such that $[k!]P$ becomes *ill-defined*?
- b. Your *failed* computation of $[k!]P$ should reveal a factor of N : What is the factor?

Hint: Failure of an inverse $1/a \bmod N$ means that $\gcd(a, N) > 1$.



Consider $N := 263629786661$ and the point $P := (2, 3)$ in the curve $E : y^2 = x^3 + x - 1$ over $\mathbb{Z}/(N) \dots$

1. a. What is the smallest positive k such that $[k!]P$ becomes *ill-defined*?
- b. Your *failed* computation of $[k!]P$ should reveal a factor of N : What is the factor?

Hint: Failure of an inverse $1/a \bmod N$ means that $\gcd(a, N) > 1$. If we *also* have $\gcd(a, N) < N$ then we have a factor!



Consider $N := 263629786661$ and the point $P := (34048322494, 57707685526)$ in the curve $E : y^2 = x^3 + 196978702920x - 239496931658$ over $\mathbb{Z}/(N)\dots$

2. a. What is the smallest positive k such that $[k!]P$ becomes *ill-defined*?
- b. Your *failed* computation of $[k!]P$ should reveal a factor of N : What is the factor?

